DATA 612 · Statistical Programming in R · Week 5

5.2 dplyr: case_when() and Distinct Rows

case_when() replaces nested if_else(). distinct() and rownames_to_column() clean identifiers.

1. `case_when()`

`case_when()` is a vectorized chain of `if` / `else`. Each argument is `condition ~ value`. Conditions are tried in order. End with `TRUE ~ ...` as the default. If nothing matches, you get `NA`.

```
x <- 1:16
case_when(
  x %% 35 == 0 ~ "fizz buzz",
  x %% 5 == 0 ~ "fizz",
  x %% 7 == 0 ~ "buzz",
  TRUE ~ as.character(x)
)
```

Put the specific cases first. A leading `TRUE ~` catches everything and the later lines never run.

`NA` in `x` is not special. Handle it with `is.na(x) ~ ...`.

Inside `mutate()` it is the usual way to build a category from several columns:

```
starwars |>
  select(name:mass, gender, species) |>
  mutate(
    type = case_when(
      height > 200 | mass > 200 ~ "large",
      species == "Droid" ~ "robot",
      TRUE ~ "other"
    )
  )
```

All right-hand sides must have the same type.

---

2. Row names and `distinct()`

Tibbles do not use row names. A matrix such as `state.x77` or `mtcars` does. `has_rownames()` checks. `rownames_to_column(mtcars, var = "car")` then `as_tibble()` turns the names into a real column.

```
has_rownames(mtcars)
mtcars |>
  rownames_to_column(var = "car") |>
  as_tibble()
```

`distinct()` keeps unique rows, in the original order. You can name columns, or use `across()`:

```
starwars |>
  distinct(across(contains("color"))) |>
  arrange(hair_color, skin_color, eye_color)
```

---

3. Programming note (aside)

dplyr verbs use **data masking** (`filter(month == 1)` means the column, not an object in the console) or **tidy selection** (`select(starts_with("dep"))`). That is why interactive work is short. Inside a function, when the column name is stored in an ordinary R object, you need extra tools. We will stay with typed column names this week.

---

4. This week, in one place

- `rowwise()` and `c_across()` for a summary across columns in one row
- `across()` to repeat a function on many columns
- `case_when()` instead of nested `if_else()`
- `distinct()` and `rownames_to_column()` when you need them

---

5. Practice

1. Check whether `mtcars` has row names. If it does, make a tibble with a `car` column.
2. Min and max of every numeric column except `birth_year`, by `species` and `gender`, plus `n()`.